

Chapter 24 - Platformer

We use the code from the previous chapter

Falling

Now that we can resolve collision we can make a platformer. A game where you fall down and can jump up. First let's create a map for us to walk around in. We can remove the single wall we added.

function love.load()

```
Object = require "classic"
```

```
require "entity"
```

```
require "player"
```

```
require "wall"
```

```
require "box"
```

```
player = Player(100, 100)
```

```
box = Box(400, 150)
```

```
objects = {}
```

```
table.insert(objects, player)
```

```
table.insert(objects, box)
```

```
map = {
```

```
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
```

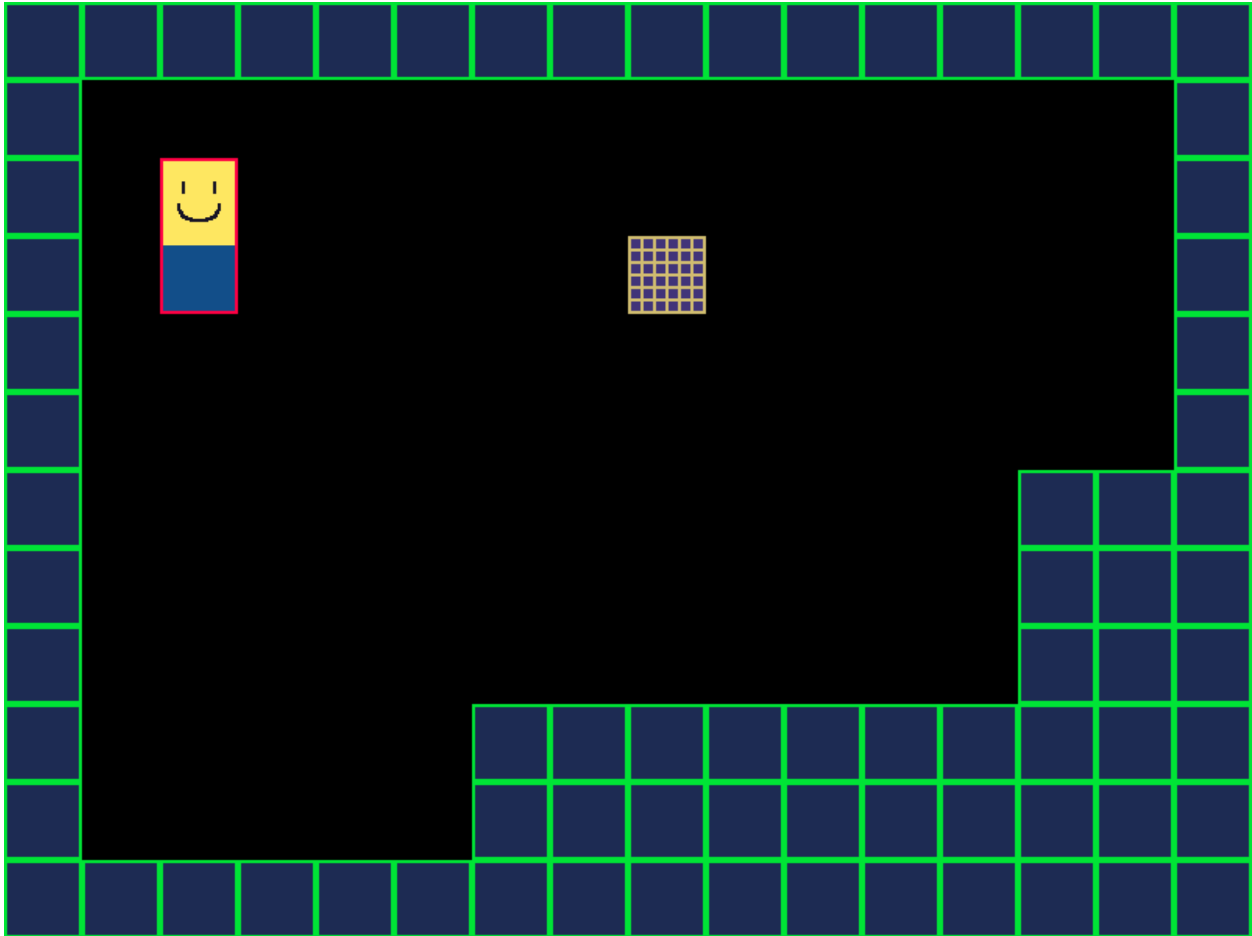
```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
{1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1},  
{1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1},  
{1,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1},  
{1,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1},  
{1,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,1},  
{1,0,0,0,0,0,1,1,1,1,1,1,1,1,1,1,1},  
{1,0,0,0,0,0,1,1,1,1,1,1,1,1,1,1,1},  
{1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
```

```
for i,v in ipairs(map) do  
  for j,w in ipairs(v) do  
    if w == 1 then  
      table.insert(objects, Wall((j-1)*50, (i-1)*50))  
    end  
  end  
end  
end
```



Depending on how good your computer is, you might notice that the game has become very slow. This is because all the walls we added are checking collision with each other. This is very inefficient, because there is no need to check this. The walls never move so they will never overlap with each other. Instead, we should create a separate table for all the walls. The objects table checks collision with itself and the walls table, but the walls table does not check for collision with itself.

function love.load()

Object = require "classic"

require "entity"

require "player"

require "wall"

require "box"

```
player = Player(100, 100)
```

```
box = Box(400, 150)
```

```
objects = {}
```

```
table.insert(objects, player)
```

```
table.insert(objects, box)
```

```
--Create the walls table
```

```
---- ADD THIS
```

```
walls = {}
```

```
-----
```

```
map = {
```

```
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,0,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,1,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,1,1},
```

```
    {1,0,0,0,0,0,0,0,0,0,0,0,0,1,1},
```

```
    {1,0,0,0,0,0,1,1,1,1,1,1,1,1,1},
```

```
    {1,0,0,0,0,0,1,1,1,1,1,1,1,1,1},
```

```
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
```

```
}
```

```

for i,v in ipairs(map) do
  for j,w in ipairs(v) do
    if w == 1 then
      -- Add all the walls to the walls table instead.
      ---- CHANGE THIS
      table.insert(walls, Wall((j-1)*50, (i-1)*50))
      -----
    end
  end
end
end

```

```

function love.update(dt)
  for i,v in ipairs(objects) do
    v:update(dt)
  end

  -- Update the walls
  ---- ADD THIS
  for i,v in ipairs(walls) do
    v:update(dt)
  end
  -----

  local loop = true

```

local limit = 0

while loop **do**

loop = **false**

limit = limit + 1

if limit > 100 **then**

break

end

for i=1,#objects-1 **do**

for j=i+1,#objects **do**

local collision = objects[i]:resolveCollision(objects[j])

if collision **then**

 loop = **true**

end

end

end

-- For each object check collision with every wall.

---- ADD THIS

for i,wall **in** ipairs(walls) **do**

for j,object **in** ipairs(objects) **do**

local collision = object:resolveCollision(wall)

if collision **then**

 loop = **true**

```

        end
    end
end
-----
end
end

```

```

function love.draw()
    for i,v in ipairs(objects) do
        v:draw()
    end

```

```

    -- Draw the walls

```

```

    ---- ADD THIS

```

```

    for i,v in ipairs(walls) do
        v:draw()
    end

```

```

-----
end

```

Okay so now we can start adding platformer physics. Let's start with falling. In player.lua we already make the player move down when we press the down-key. By removing that if-statement the player will automatically fall down.

```

--! file: player.lua

```

```

function Player:update(dt)

```

```

    Player.super.update(self, dt)

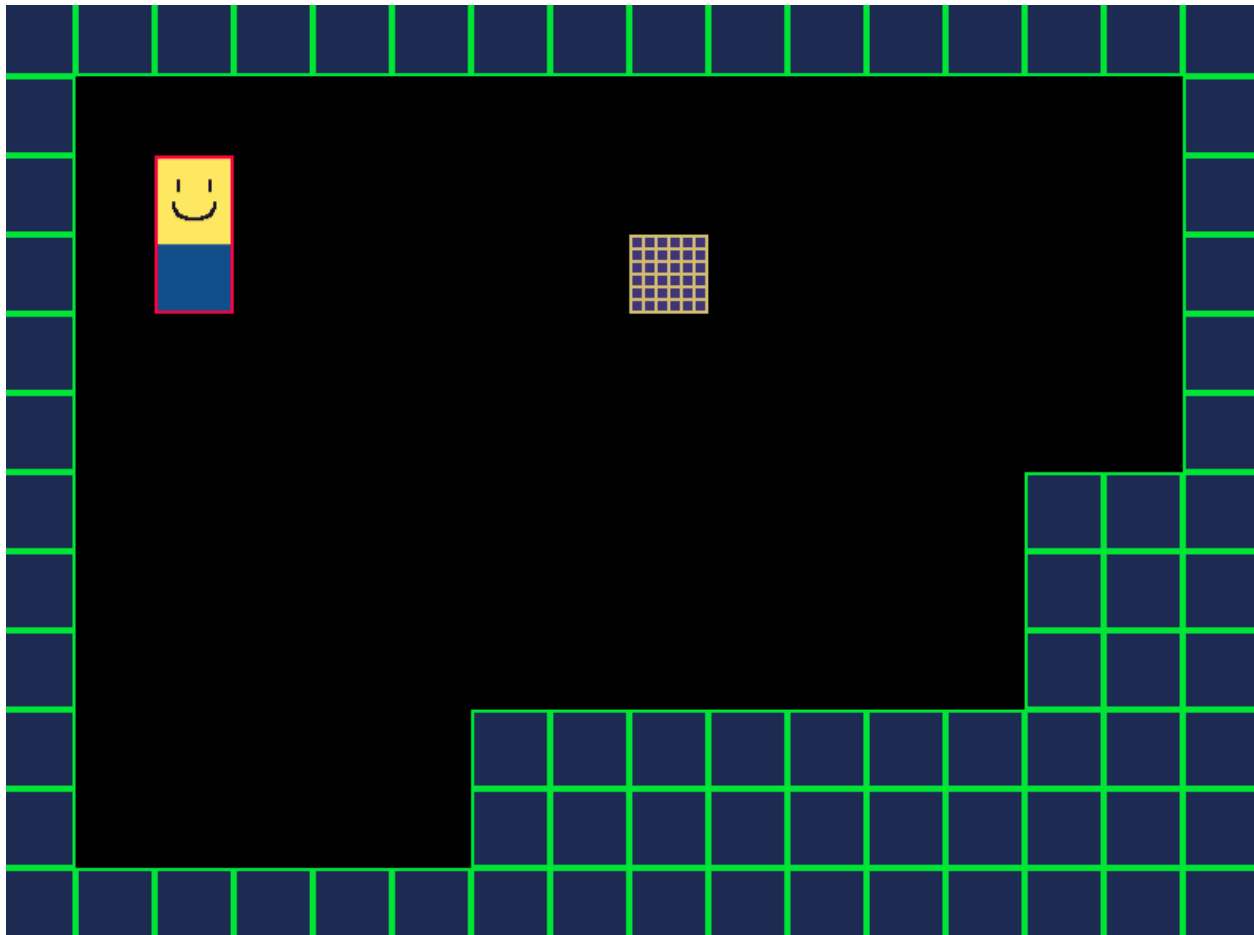
```

```

    if love.keyboard.isDown("left") then

```

```
    self.x = self.x - 200 * dt  
elseif love.keyboard.isDown("right") then  
    self.x = self.x + 200 * dt  
end  
  
if love.keyboard.isDown("up") then  
    self.y = self.y - 200 * dt  
end  
  
-- Remove the if-statement  
self.y = self.y + 200 * dt  
end
```



It works, the object falls down, but this isn't how gravity works. An object should slowly fall, and as it falls it should gain speed. Let's create something that looks more like real gravity in the Entity class. We need a gravity and weight property. We use the gravity property to increase the y-position of the entity, and we use the weight property to increase the gravity. So we increase how fast we're falling.

--! file: entity.lua

function Entity:new(x, y, image_path)

self.x = x

self.y = y

self.image = love.graphics.newImage(image_path)

self.width = self.image:getWidth()

self.height = self.image:getHeight()

```

self.last = {}

self.last.x = self.x

self.last.y = self.y


self.strength = 0

self.tempStrength = 0


-- Add the gravity and weight properties
self.gravity = 0

self.weight = 400
end


function Entity:update(dt)

    self.last.x = self.x

    self.last.y = self.y


    self.tempStrength = self.strength


    -- Increase the gravity using the weight
    self.gravity = self.gravity + self.weight * dt


    -- Increase the y-position
    self.y = self.y + self.gravity * dt
end

```

Since the walls don't need to fall, we can give it a weight of 0.

```
--! file: wall.lua
```

```
Wall = Entity:extend()
```

```
function Wall:new(x, y)
```

```
    Wall.super.new(self, x, y, "wall.png", 1)
```

```
    self.strength = 100
```

```
    self.weight = 0
```

```
end
```

And we can remove the part of the player that makes it automatically fall, as well as moving up by pressing the up-key.

```
--! file: player.lua
```

```
function Player:update(dt)
```

```
    -- It's important that we do this before changing the position
```

```
    Player.super.update(self, dt)
```

```
if love.keyboard.isDown("left") then
```

```
    self.x = self.x - 200 * dt
```

```
elseif love.keyboard.isDown("right") then
```

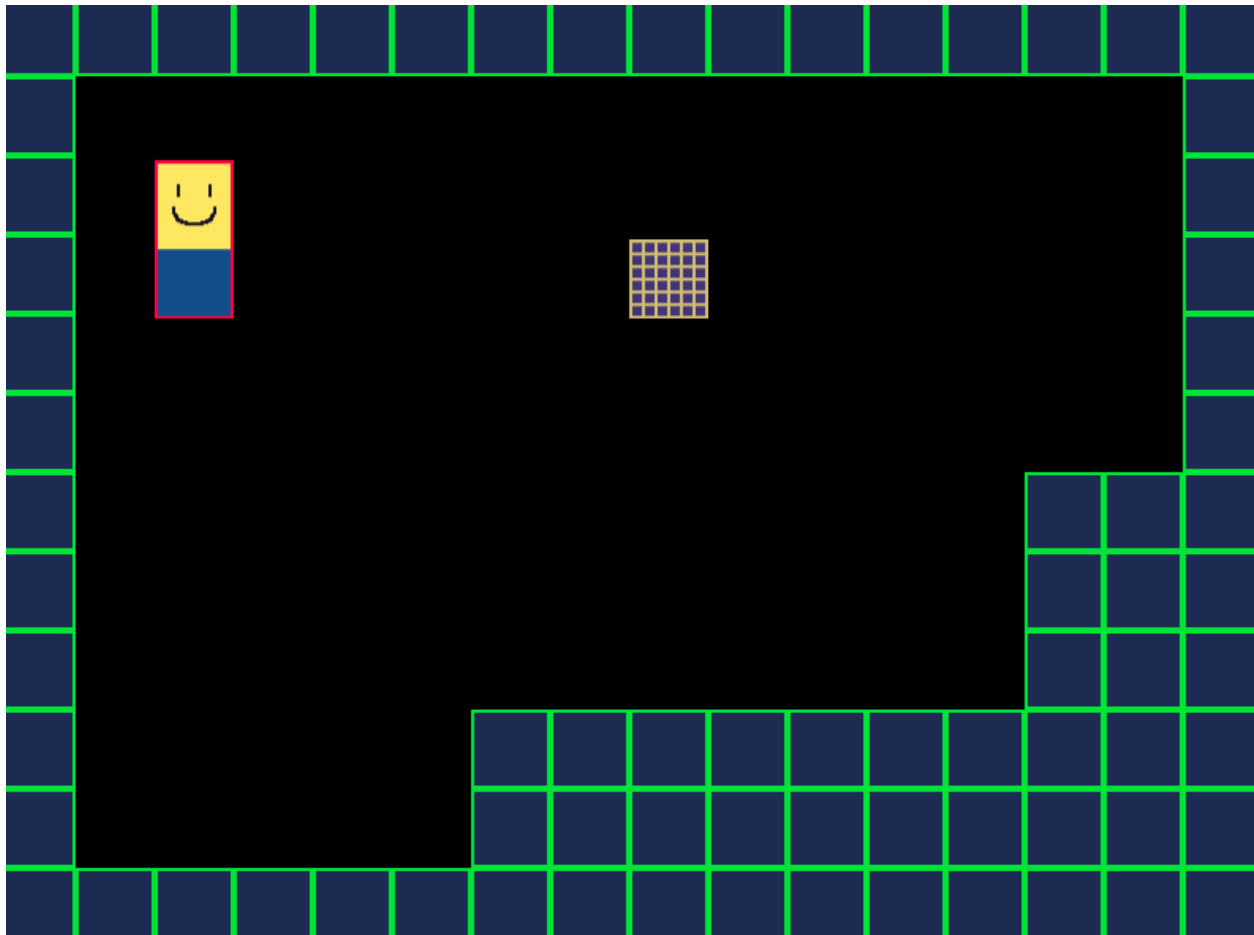
```
    self.x = self.x + 200 * dt
```

```
end
```

```
-- Remove the vertical movement
```

```
end
```

And now our player and box are falling as their fall speed increases.



But when you run the game long enough you might notice that the player and box fall right through the floor. This is because the gravity keeps increasing even though they are standing on the floor. We need to reset the gravity as they stand on the floor. We can do this in `Entity:resolveCollision(e)`.

--! file: entity.lua

function Entity:resolveCollision(e)

if self.tempStrength > e.tempStrength **then**

return e:resolveCollision(self)

end

if self:checkCollision(e) **then**

self.tempStrength = e.tempStrength

```

if self:wasVerticallyAligned(e) then

    if self.x + self.width/2 < e.x + e.width/2 then

        local pushback = self.x + self.width - e.x

        self.x = self.x - pushback

    else

        local pushback = e.x + e.width - self.x

        self.x = self.x + pushback

    end

elseif self:wasHorizontallyAligned(e) then

    if self.y + self.height/2 < e.y + e.height/2 then

        local pushback = self.y + self.height - e.y

        self.y = self.y - pushback

        -- We're touching a wall from the bottom

        -- This means we're standing on the ground.

        -- Reset the gravity

        ---- ADD THIS

        self.gravity = 0

        -----

    else

        local pushback = e.y + e.height - self.y

        self.y = self.y + pushback

    end

end

return true

end

return false

```

end

And now they don't fall through the wall anymore.

Jumping

Now it's time to make the player able to jump. We make it jump when the up-key is pressed. So first let's add the `love.keypressed(key)` callback to `main.lua`, and have it call the function `jump()` of the player, which we will make in a moment.

--! file: main.lua

function love.keypressed(key)

-- Let the player jump when the up-key is pressed

if key == "up" **then**

 player:jump()

end

end

So what needs to happen to make the player jump? It's actually very easy. We simply give the gravity a negative value. The lower the value (or in other words the more negative the value) the higher the player jumps.

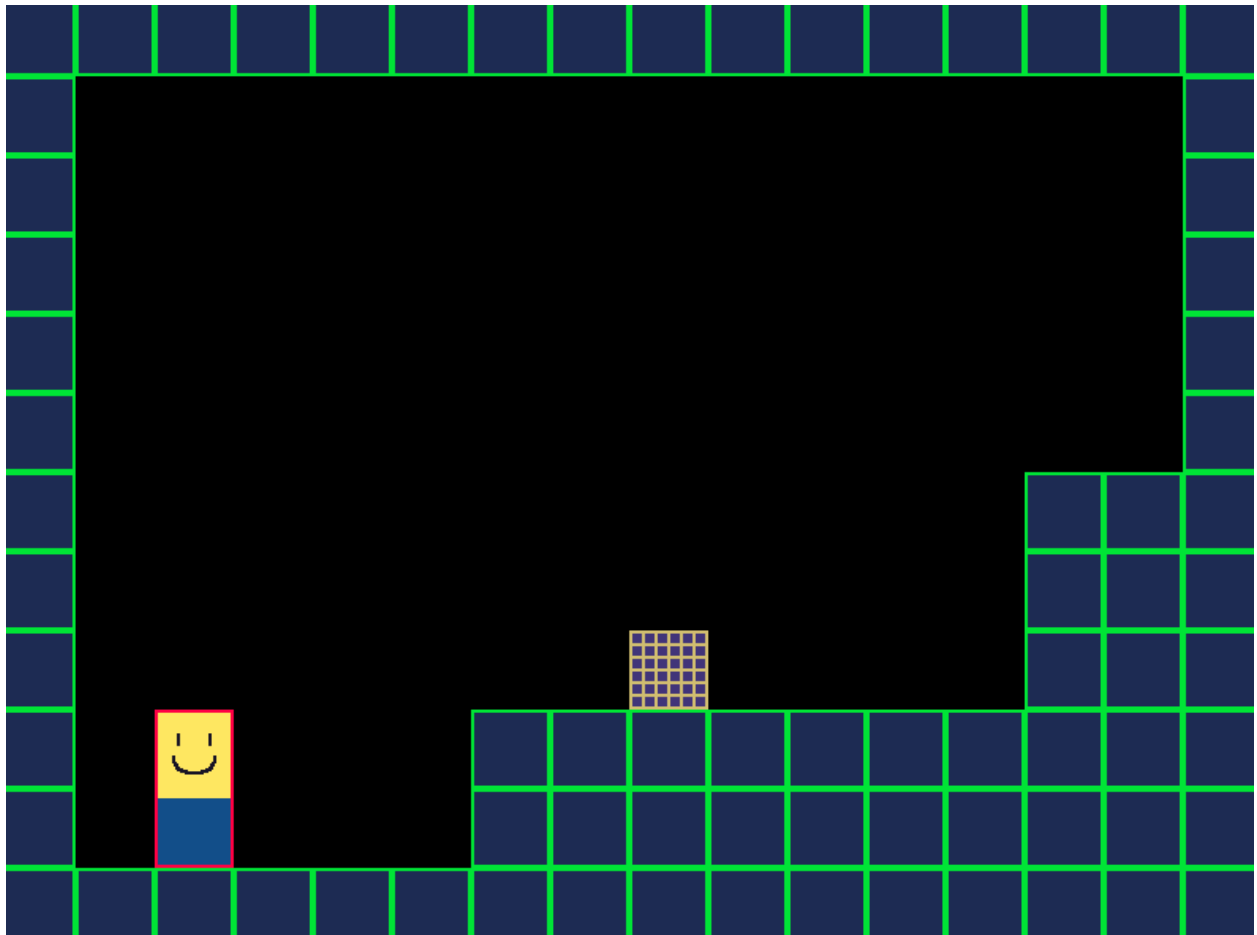
--! file: player.lua

function Player:jump()

 self.gravity = -300

end

As the player's gravity changes, it jumps in the air and slowly falls as the gravity keeps increasing.



But when you try you will notice that we can jump multiple times. We don't want that. You only should be able to jump when you stand on the ground. We can do this by adding a `canJump` property to the player. When you land on the ground the property becomes true, and when you jump, which you can only do when `canJump` is true, it becomes false.

```
function Player:new(x, y)
```

```
    Player.super.new(self, x, y, "player.png")
```

```
    self.strength = 10
```

```
    self.canJump = false
```

```
end
```

```
function Player:update(dt)
```

```
Player.super.update(self, dt)
```

```
if love.keyboard.isDown("left") then
```

```
    self.x = self.x - 200 * dt
```

```
elseif love.keyboard.isDown("right") then
```

```
    self.x = self.x + 200 * dt
```

```
end
```

```
-- Remove the if-statement
```

```
-- self.y = self.y + 200 * dt
```

```
end
```

```
function Player:jump()
```

```
    if self.canJump then
```

```
        self.gravity = -300
```

```
        self.canJump = false
```

```
    end
```

```
end
```

But right now if we want to do an action as we land, in this case setting canJump to true, we have to do it in the Entity class. Something we could technically do, but this jumping code is for the player specifically. Let's add a function that is called upon resolving collision, so that we can override those functions in the Player class.

```
--! file: entity.lua
```

```
function Entity:resolveCollision(e)
```

```
    if self.tempStrength > e.tempStrength then
```

```
        return e:resolveCollision(self)
```

```
    end
```

```

if self:checkCollision(e) then
    self.tempStrength = e.tempStrength
    if self:wasVerticallyAligned(e) then
        if self.x + self.width/2 < e.x + e.width/2 then
            -- Replace these with the functions
            self:collide(e, "right")
        else
            self:collide(e, "left")
        end
    elseif self:wasHorizontallyAligned(e) then
        if self.y + self.height/2 < e.y + e.height/2 then
            self:collide(e, "bottom")
        else
            self:collide(e, "top")
        end
    end
    return true
end
return false
end

-- When the entity collides with something with his right side

function Entity:collide(e, direction)
    if direction == "right" then
        local pushback = self.x + self.width - e.x

```

```

    self.x = self.x - pushback
elseif direction == "left" then

    local pushback = e.x + e.width - self.x

    self.x = self.x + pushback

elseif direction == "bottom" then

    local pushback = self.y + self.height - e.y

    self.y = self.y - pushback

    self.gravity = 0

elseif direction == "top" then

    local pushback = e.y + e.height - self.y

    self.y = self.y + pushback

end

end

```

And now we can override the function `collide(e)` and set `canJump` to false when direction is bottom.

--! file: player.lua

```

function Player:collide(e, direction)

    Player.super.collide(self, e, direction)

    if direction == "bottom" then

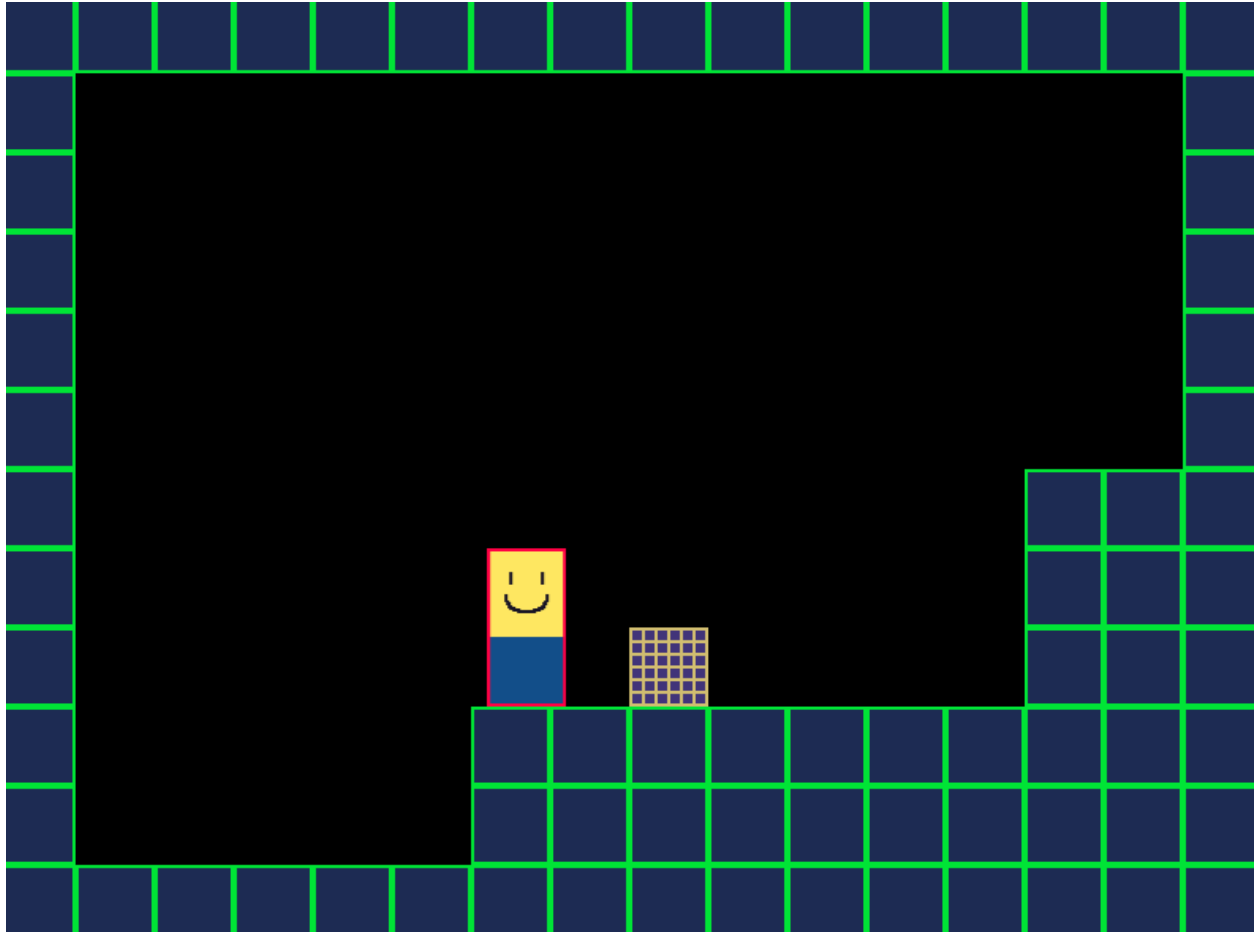
        self.canJump = true

    end

end

```

And now we can only jump once. But we still can jump mid-air when you walk off of a platform and jump.



We can fix this by checking if the previous y-position is not equal to the current y-position. When you're standing on the ground, you should not be moving vertically. So if you are it means you're not standing on the ground.

```
function Player:update(dt)
```

```
    Player.super.update(self, dt)
```

```
    if love.keyboard.isDown("left") then
```

```
        self.x = self.x - 200 * dt
```

```
    elseif love.keyboard.isDown("right") then
```

```
        self.x = self.x + 200 * dt
```

```
    end
```

```
if self.last.y ~= self.y then
```

```
    self.canJump = false
```

```
end
```

```
end
```

Nice! It's very common in platformers that you need to hit something from a certain direction. Think of Mario for example. You get only something out of the question mark when hit it from the bottom, and you can only kill an enemy by jumping on top of it. A more advanced example maybe is platforms you can jump through. When you hit it from the bottom you jump through it, but hitting it from the top makes you stand on top of it.



Let's try to create such platform.

Jump-through platform

We're going to use our box for this. We want to make it so that the player does not push the box but instead walks right through it. When it jumps on the box it stands on top of it. To make this happen we need to make some changes again. The function collide(e, direction) is for what should happen when collision is resolved. But we don't want the collision to be resolved. We want to be able to walk through the box.

How about we first check if both parties of the collision want the collision to be resolved. We create a function called checkResolved. If both self and e return true, then we continue with resolving the collision.

--! file: entity.lua

function Entity:resolveCollision(e)

if self.tempStrength > e.tempStrength **then**

return e:resolveCollision(self)

end

if self:checkCollision(e) **then**

self.tempStrength = e.tempStrength

if self:wasVerticallyAligned(e) **then**

if self.x + self.width/2 < e.x + e.width/2 **then**

-- Call checkResolve for both parties.

local a = self:checkResolve(e, "right")

local b = e:checkResolve(self, "left")

-- If both a and b are true then resolve the collision.

if a **and** b **then**

self:collide(e, "right")

end

else

local a = self:checkResolve(e, "left")

local b = e:checkResolve(self, "right")

```

        if a and b then
            self:collide(e, "left")
        end
    end

elseif self:wasHorizontallyAligned(e) then
    if self.y + self.height/2 < e.y + e.height/2 then
        local a = self:checkResolve(e, "bottom")
        local b = e:checkResolve(self, "top")
        if a and b then
            self:collide(e, "bottom")
        end
    else
        local a = self:checkResolve(e, "bottom")
        local b = e:checkResolve(self, "top")
        if a and b then
            self:collide(e, "top")
        end
    end
end

return true
end

return false
end

function Entity:checkResolve(e, direction)
    return true
end

```

end

Now in `player.lua` we can override the function `checkResolve(e, direction)`. With classic, the class library we're using, every class has the function `is:(class)` which can be used to check if an instance of a class is of a certain type of class. So we can use `e:is(Box)` to check if `e` is of the type `Box`. This also works with base classes. So if `e` were to be a box, then `e:is(Entity)` would return true, since `Box` is an extension of the base class `Entity`.

We first check if we're colliding with a `Box`, and if so, we check if the direction is "bottom". If so, return true (meaning we want collision to be resolved), else return false.

--! file: player.lua

function Player:checkResolve(e, direction)

if e:is(Box) then

if direction == "bottom" then

return true

else

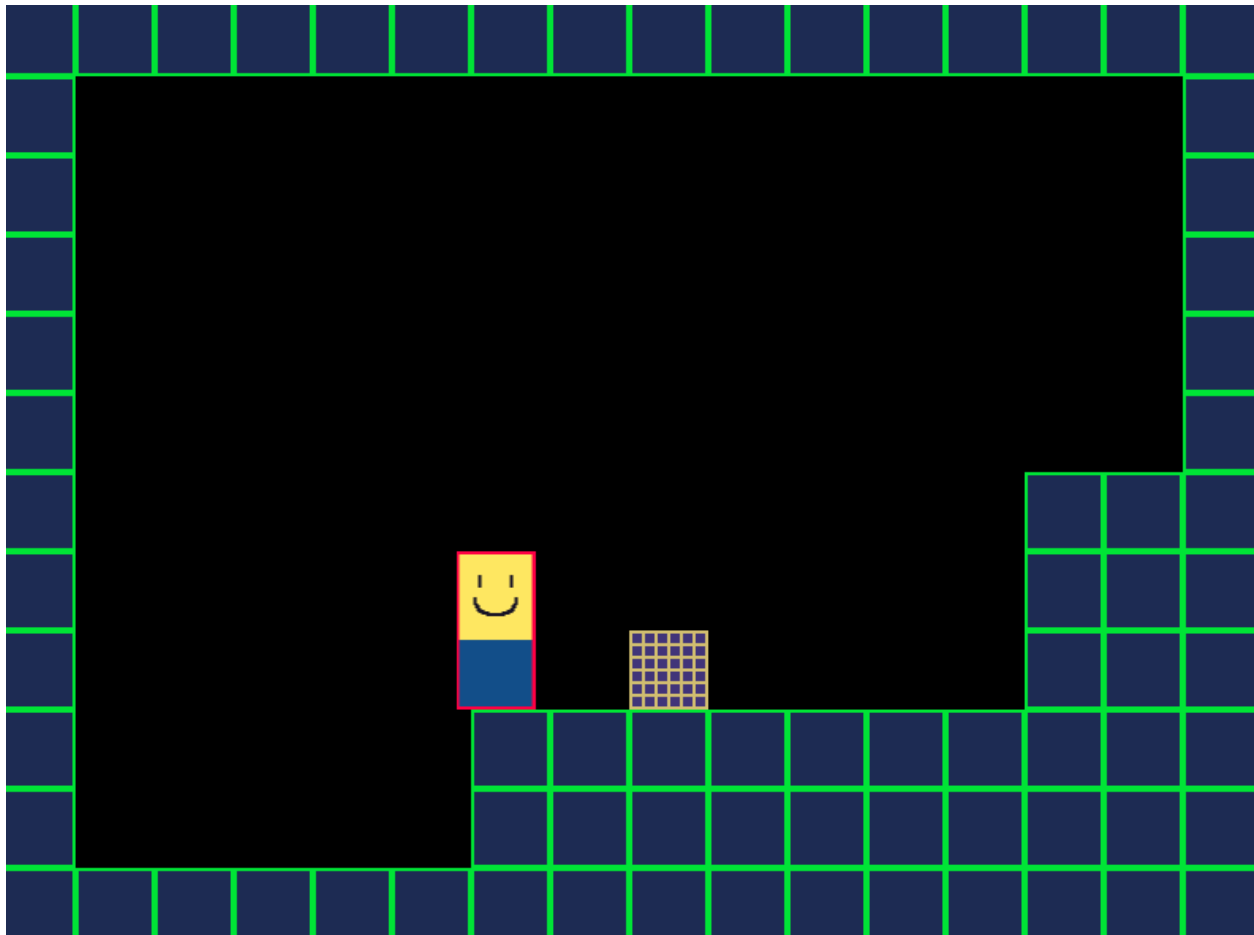
return false

end

end

return true

end



Summary

By increasing the value that we use to increase our y-position we can simulate gravity. We can jump by setting the gravity to a negative value. By adding functions and overriding those functions we can apply actions upon collision, or prevent collision from happening.

The end?

And with that we're at the end of this chapter, and at the end of this book (for now!). I hope you enjoyed reading all the chapters and of course learned a lot by reading them. I still have plans for new chapters, but that is for another time. Until then I wish you good luck on your journey to become an even greater game programmer than you already are. I recommend [learn2love](https://www.learn2love.com/) if you want to learn more. It goes more in-depth on the topics I covered, and also has chapters on networking.

Remember to practice a lot. Like I said in the introduction, you can read about painting all you want, but to learn it you have to do it. Same goes for programming. Cheers!